Official Rules and Regulations

International Cupadi Control Competition (ICCC)

ICCC Organizing Committee

November 1, 2026

1 Introduction

The International Cupadi Control Competition (ICCC) is a public platform designed to bridge the gap between theoretical control systems and practical software engineering. Competitors study the nonlinear model of a dynamic system, write a control law for it, watch the closed-loop response in an interactive simulator, and submit the law to that system's leaderboard. The current edition features eight benchmark problems, from the classic cart-pole to a quadcopter balancing a pole (Section 5).

2 Competition Categories

Participants may enter the competition under one or more of the following specialized categories, numbered according to difficulty:

1. **Web or Networked Control:** Control laws written, tested and submitted on the native web-based platform. The models run in simulation; real systems will follow.

2. **AI Control:** Application of Machine Learning (ML) to derive control laws. The control law in this category must be a data-driven ML model, such as a trained neural network whose weights are written into the law. All the systems will be in a simulated environment, at first.

3. **Digital Control:** Discrete-time implementations, focusing on sampling rates and quantization effects. This is the category most control engineers have experience with: the objective is to use a programming language to formulate a control law executed on a computer connected to the real system.

4. **Analog Control:** Continuous-time control and classical controller design. This category is the "real deal", the **old school** way of building control systems and the *hardest* of the competition: only mechanical or electronic components may be used to control the system.

This edition. Categories 1 and 2 are open: entries are control laws submitted on the web platform and scored in simulation. Categories 3 and 4 need the physical systems; their rules and dates will be announced separately. The platform ranks all the entries for a system together; to enter a result in the AI Control category, also send its documentation (Section 4) and state the category.

3 Technical Requirements & Participation

To compete, competitors must use the official platform and channels of communication (Section 6).

1. **Platform Access:** All simulations and submissions must be performed on the web platform. Each system has its own tab with its model, a code editor, plots of the response, and its leaderboard.

2. **Implementation:** Control laws must be written in JavaScript syntax within the provided Interactive Code Editor. The code is the body of a function that reads the system's variables and sets its inputs, which start at 0 on every call. For the 2D Inverted Pendulum:

```
var K1 = 1, K2 = 1, K3 = 1, K4 = 1;  // your gains
u = K1*x + K2*xdot + K3*theta + K4*thetadot;
```

3. **State Variables:** The law receives the full, exact state (no noise or delay), the elapsed time `t` (s), and `mem`, an object that keeps whatever the law stores in it during a run and starts empty on every run. JavaScript's `Math` functions are available.

4. **Sampling:** The law is called once per time step, at the system's rate (Section 5). Its inputs are held constant until the next call, and the model is integrated with the fourth-order Runge–Kutta method.

5. **Input Limits:** Every input must be a finite number, or the run fails. Inputs beyond a system's limits are clamped to them; the 2D Rocket Landing has its own rule.

6. **Server Limits:** On submission, the server re-runs the law in a sandboxed JavaScript interpreter with no network or file access. Over the whole run, the law may use at most 500 ms of computing time and 8 MB of memory. The interpreter is slower than a browser, so a law that runs in the browser may still exceed the time limit on the server. A submission, including name and email, must not exceed 10 kB, which limits a law to about 9,500 characters.

4 Scoring and Leaderboard

Every run is scored automatically with a cost that penalizes the distance from the goal and the control effort, so the best design is a well-tuned controller rather than the one with the largest gains. For every system except the 2D Rocket Landing,

$$\text{Score} = \max\left(0,\ 1000 - k\int_0^T \ell\, dt\right), \tag{1}$$

where the weight k, the duration T and the running cost ℓ are given for each system in Section 5. The integral is the sum, over the time steps, of $\ell\,\Delta t$, with ℓ evaluated at the end of each step using the input applied during it. Scores keep two decimals.

- **Failure:** A run that meets its system's failure condition fails. A failed run scores 0 and cannot be submitted.

- **Official Score:** On submission, the server re-runs the control law and records its own score; the score shown in the browser is a preview. Only scores computed by the server count. Entries submitted before server verification was introduced, and entries for systems no longer offered, are not ranked; to rank such a law, submit it again.
- **Leaderboards:** Each system has its own leaderboard, shown in its tab: the top 10 competitors, each with their best score. Competitors are identified by their email address, and ties go to the earlier submission. Emails are never shown publicly. Competitors may submit as often as they want, up to 10 submissions per minute.
- **Documentation:** Users are encouraged to send a PDF to the official email documenting their mathematical models and control signals for validation. The platform's *Print* button produces a report with the model, the control law and the plots, which the browser can save as a PDF. Only documents in the PDF format will be evaluated!
- **Validation:** All results must be repeatable. The simulation is deterministic, so the same law always produces the same score; a law that uses randomness may score differently on the server, and the server's score is the one that counts.

5 The Systems

Each system's tab gives its equations of motion, parameters and sign conventions. The simulator's source code, shared by the browser and the server, is available at iccc.lesaf.cc/js/sim.js. Angles are in radians; in 3D, they are ZYX Euler angles (roll ϕ, pitch θ, yaw ψ). Participants may use whatever techniques they want: the competition is about strategy and information.

5.1 2D Inverted Pendulum

The cart-pole: a point mass on a massless pole, without friction, from Tedrake, *Underactuated Robotics*, Ch. 3, with θ measured from upright. The objective is to design a controller u such that

$$\lim_{t\to\infty} \theta(t) = 0 \quad \text{and} \quad \lim_{t\to\infty} x(t) = 0. \tag{2}$$

- **Reads** `x, xdot, theta, thetadot`; **sets** `u`, the cart force, limited to ±1000 N.
- **Start:** at rest at $x = 0$, with $\theta = 0.1$ rad.
- **Run:** 10 s at 100 Hz. **Fails if** $|\theta| > \pi$.
- **Score:** $k = 50$, $\ell = x^2 + \theta^2 + 0.1\,u^2$.

5.2 2D Inverted Pendulum, Swing-up Mode

The same cart-pole, starting hanging down, on a finite track and with a realistic force limit. It is a mode of the pendulum's tab, with its own leaderboard. From $\theta(0) = \pi$, the objective is to design a controller u such that

$$\lim_{t\to\infty} \theta(t) = 0 \quad \text{and} \quad \lim_{t\to\infty} x(t) = 0, \quad \text{with } |x(t)| \le 2 \text{ m for all } t. \tag{3}$$

- **Reads** `x, xdot, theta, thetadot`, with θ wrapped to $(-\pi, \pi]$, so upright is still $\theta = 0$; **sets** `u`, limited to ± 10 N.
- **Start:** at rest at $x = 0$, hanging straight down ($\theta = \pi$).
- **Run:** 10 s at 100 Hz. **Fails if** the cart leaves the ± 2 m track, or if the pendulum does not end within 0.2 rad of upright.
- **Score:** $k = 25$, $\ell = x^2 + \theta^2 + 0.01\,u^2$.

5.3 2D Acrobot

Two links on a fixed pivot with a motor only at the elbow, in the manipulator equations of Tedrake, *Underactuated Robotics*, Ch. 3, with the link parameters of Spong (IEEE Control Systems, 1995). From hanging down, the objective is to design a controller u such that

$$\lim_{t\to\infty} \theta_1(t) = 0 \quad \text{and} \quad \lim_{t\to\infty} \theta_2(t) = 0. \tag{4}$$

- **Reads** `theta1, theta2, theta1dot, theta2dot`, both angles from upright and wrapped to $(-\pi, \pi]$; **sets** `u`, the elbow torque, limited to ± 30 N·m.
- **Start:** at rest, hanging straight down ($\theta_1 = \pi$, $\theta_2 = 0$).
- **Run:** 15 s at 100 Hz. **Fails if** either angle does not end within 0.2 rad of upright.
- **Score:** $k = 10$, $\ell = \theta_1^2 + \theta_2^2 + 0.1\,(\dot\theta_1^2 + \dot\theta_2^2) + 0.01\,u^2$.

5.4 2D Ballbot

The planar (X–Z) model of the Ballbot_LMI_V2 prototype built at ITA, with its real parameters: a body balancing on a ball driven by omniwheels. The objective is to design a controller T such that

$$\lim_{t\to\infty} \theta(t) = 0 \quad \text{and} \quad \lim_{t\to\infty} x(t) = 0. \tag{5}$$

- **Reads** `x, theta, xdot, thetadot`; **sets** `T`, the motor torque, limited to ± 1 N·m.
- **Start:** at rest at $x = 0.3$ m, with $\theta = 0.1$ rad.
- **Run:** 10 s at 200 Hz. **Fails if** $|\theta| > \pi/2$.
- **Score:** $k = 100$, $\ell = 0.1\,x^2 + 10\,\theta^2 + \dot x^2 + \dot\theta^2 + T^2$, the weights of the prototype's controller design.

5.5 2D Rocket Landing

Powered descent in the plane, after Açıkmeşe & Ploen (JGCD, 2007): their Mars lander, a point mass whose engine never shuts off while there is fuel. The objective is to design the thrust $\mathbf{T} = (T_x, T_z)$ that lands on the pad, at a time t_f, with the least fuel:

$$\begin{aligned} &\min_{\mathbf{T}}\ m(0) - m(t_f) \\ &\text{s.t. } z(t_f) = 0, \quad |x(t_f)| \le 10 \text{ m}, \quad \|\mathbf{v}(t_f)\| \le 2 \text{ m/s}, \quad t_f \le 120 \text{ s}, \\ &\qquad z(t) \ge \tan 4^\circ\,(|x(t)| - 10 \text{ m}), \quad \rho_1 \le \|\mathbf{T}(t)\| \le \rho_2. \end{aligned} \tag{6}$$

- **Reads** `x, z, vx, vz, m` (pad at $x = 0$, ground at $z = 0$); **sets** `Tx, Tz`, the net thrust (N). Its direction is kept and its magnitude pushed into $[\rho_1, \rho_2] = [4972, 13258]$ N; a zero command fires straight up at the minimum, and an empty tank ($m = 1505$ kg) shuts the engine off.

- **Start:** 2 km downrange and 1.5 km up, approaching at $(v_x, v_z) = (-100, -75)$ m/s, with $m = 1905$ kg.
- **Run:** 20 Hz, until touchdown ($z \le 0$). **Fails if** the lander touches down faster than 2 m/s or more than 10 m from the pad, dips below the 4° glide slope $z \ge \tan 4° \, (|x| - 10)$, or has not touched down after 120 s.
- **Score:** by the fuel used, $\min(1000,\ 1000\, m_{\text{opt}}/m_{\text{used}})$, where $m_{\text{opt}} = 204.6$ kg is the minimum fuel for this landing, computed with the paper's convex program.

5.6 3D Ballbot

The 2D Ballbot in three dimensions: the ball rolls in the plane while the body rolls, pitches and yaws. In each plane, the model reduces exactly to the prototype's planar models. The objective is to design the torques T_x, T_y, T_z such that

$$\lim_{t\to\infty} \big(x(t), y(t)\big) = \mathbf{0} \quad \text{and} \quad \lim_{t\to\infty} \big(\phi(t), \theta(t), \psi(t)\big) = \mathbf{0}. \tag{7}$$

- **Reads** `x, y, phi, theta, psi, xdot, ydot, phidot, thetadot, psidot`; **sets** `Tx, Ty, Tz`, the torques about the body's axes, each limited to ±1 N·m.
- **Start:** shoved: at $(x, y) = (0.3, -0.2)$ m moving at $(-0.6, 0.4)$ m/s, tilted $(\phi, \theta) = (-0.08, 0.1)$ rad, yawed $\psi = 1.2$ rad and spinning at 2 rad/s.
- **Run:** 10 s at 200 Hz. **Fails if** the body tilts past horizontal.
- **Score:** $k = 100$, $\ell = 0.1\,(x^2{+}y^2){+}10\,(\phi^2{+}\theta^2){+}\dot{x}^2{+}\dot{y}^2{+}\dot{\phi}^2{+}\dot{\theta}^2{+}\psi^2{+}0.1\,\dot{\psi}^2{+}T_x^2{+}T_y^2{+}T_z^2$.

5.7 3D Quadcopter

The quadrotor model of Heeg, Song & Scaramuzza (arXiv:2410.15979, 2024), commanded by collective thrust and body rates. With position $\mathbf{p} = (x, y, z)$ and velocity $\mathbf{v} = (v_x, v_y, v_z)$, the objective is to design c and $\boldsymbol{\omega} = (\omega_x, \omega_y, \omega_z)$ such that the quadcopter hovers at the origin:

$$\lim_{t\to\infty} \mathbf{p}(t) = \mathbf{0} \quad \text{and} \quad \lim_{t\to\infty} \mathbf{v}(t) = \mathbf{0}. \tag{8}$$

- **Reads** `x, y, z, vx, vy, vz, phi, theta, psi` (z up); **sets** `c`, the mass-normalized collective thrust, from 0 to $3g = 29.43$ m/s², and the body rates `wx, wy` (±6 rad/s) and `wz` (±2 rad/s). Hovering needs $c = g = 9.81$ m/s².
- **Start:** thrown from $\mathbf{p} = (-1, 0.8, -0.5)$ m at $\mathbf{v} = (1.5, -1, 2)$ m/s, with $(\phi, \theta, \psi) = (0.5, -0.4, 0.8)$ rad.
- **Run:** 10 s at 100 Hz. **Fails if** it hits the ground at $z = -2$ m.
- **Score:** $k = 10$, $\ell = 0.2\, L_H(5\mathbf{p}) + 0.1\, L_H(\mathbf{v}) + 0.1\, L_H(\boldsymbol{\omega}) + 0.5\, L_H(\mathbf{u} - \mathbf{u}_{\text{hover}})$, the paper's reward terms, where L_H is the Huber loss ($\delta = 1$) summed over components, $\mathbf{u} = (c, \boldsymbol{\omega})$ and $\mathbf{u}_{\text{hover}} = (g, 0, 0, 0)$.

5.8 3D Flying Inverted Pendulum

The 3D Quadcopter with a pole balanced on top, after Hehn & D'Andrea (ICRA 2011): a point mass on a massless 0.8 m rod. The pole does not affect the vehicle. The objective is to design c and $\boldsymbol{\omega}$ such that the vehicle hovers at the origin with the pole upright:

$$\lim_{t\to\infty} \mathbf{p}(t) = \mathbf{0}, \quad \lim_{t\to\infty} \big(r(t), s(t)\big) = \mathbf{0} \quad \text{and} \quad \lim_{t\to\infty} \psi(t) = 0. \tag{9}$$

- **Reads** the 3D Quadcopter's variables plus `r, s, rdot, sdot`, the pole's horizontal offset from the vehicle (m) and its rates (m/s); **sets** the 3D Quadcopter's inputs, with the same limits.
- **Start:** at rest at $\mathbf{p} = (0.8, -0.6, 0.3)$ m, with $\psi = 0.3$ rad and the pole at $(r, s) = (0.1, -0.08)$ m.
- **Run:** 10 s at 100 Hz. **Fails if** the pole tilts past 60° or the vehicle hits the ground at $z = -2$ m.
- **Score:** $k = 50$, $\ell = |\mathbf{p}|^2 + 10\,(r^2 + s^2) + 0.1\,(|\mathbf{v}|^2 + \dot{r}^2 + \dot{s}^2) + \psi^2 + 0.01\,\big((c - g)^2 + |\boldsymbol{\omega}|^2\big)$.

6 Contact and Resources

- **Official email:** lesaf@ita.br
- **Web platform** (simulation, submissions and leaderboards): iccc.lesaf.cc